Kendall Beam - beamk@hawaii.edu - ICS 385 - Big Island

Product Requirements Document (PRD) v3.0 - May 1, 2026 – Week 15 Final

Hilo Eco-tourism Rental

Term Project 3 - Full Stack Website - Spring 26

OVERVIEW

Problem statement: A website app to service (marketing, booking, contact) for a vacation rental property in Hilo, Hawaii.

Property/Audience: The target market is eco-tourism due to the close proximity to Kilauea, Mauna Loa, Mauna Kea, and beaches along the Big Island's windward coastline. Hilo was chosen (rather than a location closer to Kilauea) because of the amenities found in the larger town, hopefully opening up to a broader range of customers.

Website Requirements: The site will consist of three/four pages:
R1) a) a marketing page that includes a hero section, b) a dashboard for displaying charts/stats (from stored records) and current weather (from an API), c) amenities of the property listing, d) a way to get to the booking page.
R2) a booking page/modal houses a form to inquire for more information, an email is sent to admin accounts. (booking capabilities can be outsourced)
R4) and an Admin page for updating content.
R5) a dashboard that presents relevant information to the user (Weather, property location, kilauea eruption frequency, current kilauea conditions)

UX/ CSS Design –
R5) All pages will include responsive design (grid/flexbox, UX feedback), be accessible (proper headers, alt-text, >4.5 contrast, mobile-friendly at 375px), and keep a consistent design to invoke similarity.

Technical Architecture – The app will use the following:
1) React for rendering HTML/CSS in a dynamic way client-side,
2) a JavaScript Node.js environment running Express.js for handling backend routes/ REST API needs,
3) MongoDB/Mongoose.js for storing data (DBEDT records, property info, reviews, bookings) and

data modeling,

4) Chart.js/D3.js for data visualization in dashboard,

5) authentication using passport.js to manage admin page credentials.

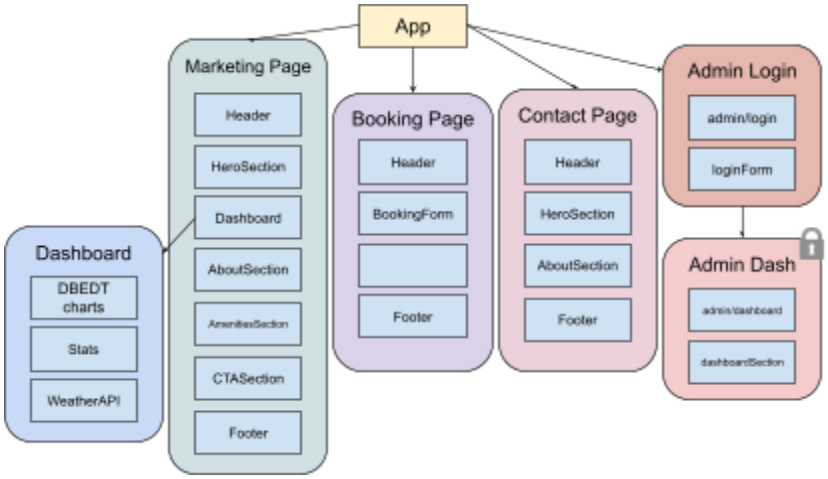
Security *should be a concern throughout (no hardcoded/public secrets, input sanitization, HTTP security, CORS security, and password/login protections at the minimum).*

User Personas	John Doe Visitor, looking housing options, to see Kilauea	Jane Who Already booked, confirming reservation, planning trip	Operator adding/ updating content, fit into busy schedule
Information/ Functionality/ Wants	- good first impression - easy to read info - simple user flow - local recommendations (activities, food) - wifi? home rules? parking? costs? bedrooms?	- easy to find booking info - see status, add to calendar - what amenities are with property - 'springboard' to find things to do nearby - how to get to things nearby?	+ easy to use UI + easy to use login (reset password?) + see what is currently posted + age of information (since last update) + privileged access unique to an admin role
Considerations / Goals/ Needs	- reaching out for help? - why this place over somewhere else? - disability accommodations? - hotel level quality at a discount	- know who the user is - update /cancel booking? - further links to amenities if possible	+ update content — add/remove property/packages — update property info + see backend status — is db running, console errors + see website stats — # of bookings and when — page visits, etc.
Possible frustrations	- is the owner trustworthy? - Bad UX? Too easy? - others' reviews?	- dont know password - avoid information overload - bad actors modifying data if unsecure route exists	+ no web development experience + avoid overly complicated + dont know password + admin access gets compromised

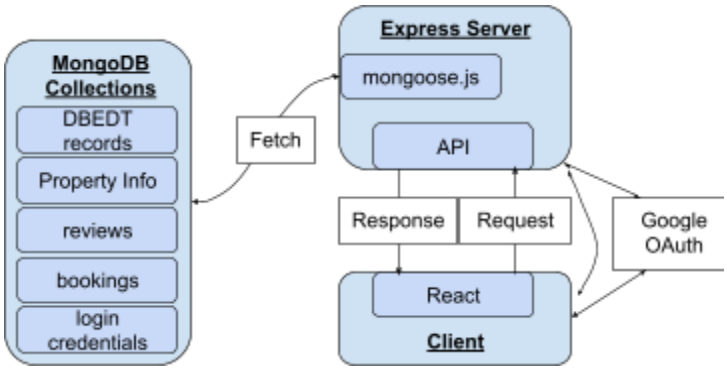
Component Requirements

<table>
<tr><td>Marketing Page</td><td>
<table>
<tr><td>Header</td><td>HeroSection</td><td>Dashboard</td><td>About Section</td><td>Amenities Section</td><td>CTASection</td><td>Footer</td></tr>
<tr><td>– logo is anchor to home page
– links to other pages</td><td>– image of house/kilauea
– “address” title
– “vacation rental” subtitle</td><td>– DBEDT charts
+ (arrivals [month/year]
+ visitor origin (pie)
+ other stats
– current weather (via api)</td><td>– property description
+ geo-tourism
– property location</td><td>– list of amenities</td><td>– [Book Now] button
– Contact Information</td><td>– links to other pages
– copyright, year</td></tr>
</table>
</td></tr>
<tr><td>Booking Page/ Modal</td><td>
<table>
<tr><td>Login Form (in Modal)</td></tr>
<tr><td>Email, Password, Submit button</td></tr>
</table>
</td></tr>
<tr><td>About Page</td><td>
<table>
<tr><td>Website Header</td><td>About HeroSection</td><td>AboutSection</td><td>Website Footer</td></tr>
<tr><td>– logo is anchor to home page
– links to other pages</td><td>– “About/Contact” title
– different image</td><td>– owner image
– owner background
– owner contact</td><td>– links to other pages
– copyright, year</td></tr>
</table>
</td></tr>
<tr><td>Admin</td><td>
<table>
<tr><td>Admin/login</td><td>Admin/dashboard</td></tr>
<tr><td>– login page
(stores a session for 7 days)</td><td>– update property/package information
(options = backend status, site stats, is info old?)</td></tr>
</table>
</td></tr>
</table>

App -> child diagram



Data Flow



APIs that are exposed:

/properties	GET /properties? – (returns JSON list of filterable properties) GET /properties:id – (returns JSON of info of one property by its MongoDB id) POST properties/:id/reviews – (add review to database)
/auth	GET admin/login - render react login page POST admin/login - serialize login attempt GET admin/logout - signout, redirect to homepage
/admin (protected)	If authorized – GET admin/dashboard react page POST admin/updates – update content (post add/remove property/package)

Authentication and Security

Authentication Requirements –

Method: username + password using a local strategy (passport.js), store credentials in password database (MongoDB). Passwords are salted to prevent misuse of authorization if compromise happens. Makes authorization easier.

Method: Google OAuth where the user is redirected to Google Sign In, authentication is passed back to the server for a persistent session. This lifts the authentication burden from the backend towards a trusted source.

Admin User/Model: maintainer of site content. Can view the admin dashboard for updating page content / seeing relevant backend information. Authentication happens either through a googleId match or a username + password match. A session is created and access to the protected admin/dashboard route is authorized. Admin Users need to be created through a seed script on the back end to have authorization.

Logic Flow: admin login form is submitted, Post request is sent to server, credentials are compared to what is stored. If passed, then loads admin/dashboard. If failed, then prompt an error message and log the erroneous attempt.

Session Management: trust is saved for 7 days. If inactive for > 1 hour, then log out.

Protected Routes: /admin/dashboard needs passport's isAuthenticated() to return true to access.

Secrets Management: Backend secrets are housed in .env file (MONGO_URI, SESSION_SECRET, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET), client API key is bundled with Vite build (VITE_OPENWEATHER_KEY). .env is in .gitignore.

Threats the design mitigates:

OWASP 01 Broken access control – admin dashboard route is protected with isAdmin.js middleware checking for role = 'admin'.

OWASP 03 Injection - user dependent NOSQL queries (/login, /register) are sanitized before they are run

OWASP 07 Identification and Authentication Failures - Passport, bcrypt verification, session serialization, logout, and Google OAuth work in tandem to harden the backend.

(Partial) OWASP 02 - passwords are hashed and sessions are server-side, but cookies are not secure

Acceptance Criteria (

AC-1 Public marketing page
GIVEN any visitor / WHEN they request GET / / THEN the marketing page returns HTTP 200 and renders the property name, hero image, and at least 3 amenities.

AC-2 Visitor dashboard
GIVEN any visitor / WHEN they request GET /dashboard / THEN three Chart.js

visualizations are rendered with non-empty data.

AC-3 Local sign-up
GIVEN a new e-mail / WHEN POST /register with valid e-mail and password ≥ 8 chars / THEN HTTP 302 to /admin/dashboard, the users collection contains the new document, password is bcrypt-hashed (not plaintext).

AC-4 Local sign-in
GIVEN a registered user / WHEN POST /login with correct credentials / THEN HTTP 302 to /admin/dashboard and a session cookie is set.

AC-5 Google OAuth sign-in
GIVEN any user with a Google account / WHEN they click "Sign in with Google" and consent / THEN HTTP 302 to /admin/dashboard, users collection contains a document with googleId and provider='google'.

AC-6 Protected route guard
GIVEN an unauthenticated visitor / WHEN they request /admin/dashboard / THEN HTTP 302 redirects to /login and no admin content is rendered.

AC-7 Logout
GIVEN an authenticated user / WHEN they POST /logout / THEN session cookie is cleared and req.user is undefined on the next request.

AC-8 Secret hygiene
GIVEN the deployed application / WHEN inspecting the GitHub repository / THEN .env is absent, .env.example is present, and no API keys appear in any file.

```
git log --all --full-history -- .env ## blank output == good
```

AC-9 Load Time
GIVEN any visitor / WHEN they load any page / THEN it takes no longer than 500 ms to load most of the content.

AC-10 Booking Flow
GIVEN a visitor that wants to make a booking / WHEN they try to

```
kendallbeam@FBI-Apple-Device-15 term-project14 % npm run test:acceptance
 PASS  tests/auth.test.js
  Auth acceptance tests (AC-3..AC-7)
    ✓ AC-3 Local sign-up: POST /register creates user and redirects (449 ms)
    ✓ AC-4 Local sign-in: POST /login sets session cookie and redirects (428 ms)
    ✓ AC-6 Protected route guard: unauthenticated GET /admin/dashboard redirects to /admin/login (2 ms)
    ✓ AC-5 Google OAuth sign-in: simulated callback creates google user and redirects (3 ms)
    ✓ AC-7 Logout: POST /logout clears session (431 ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        1.825 s, estimated 2 s
Ran all test suites.
```

AI use: Jest + supertests were generated by GPT5.3 Codex, so validity should be taken with a grain of salt.